Abstract

This document provides the necessary background information on the calibration methods and the software applications used, with the ultimate goal of producing a guide for the programmers who wish to either modify or extend the capabilities of the software suite developed. This guide is an attempt at making life easier for the user by describing how each step in the calibration procedure has been implemented and how each device connection has been handled. The project's aim is to develop an automated measurement system for characterizing the Renesas FS-1012-1100-NG transducer. This thermal mass flow transducer's output is a voltage signal. This signal has to be analysed and compared with the measurement produced by a pre-calibrated transducer of similar characteristics. For this aim, the Renesas FS-2012-1100-NG was chosen. The task involves several steps, which are all dealt with using a unique software application developed within the LabVIEW development environment: the main VI. Nevertheless, this program calls several other sub-routines to leverage multithreading and parallel programming to handle multiple device connections simultaneously, and to manage each user request in response to a GUI event. Finally, the project's main software application includes a set of data analysis tools, which bring up the statistical distribution of samples in each data set acquired, reveal the presence of Hysteresis, and compute the combined standard uncertainty resulting from a set of uncertainty sources within the measuring system itself.

Created in LABVIEW 2021

21

"Programmer's Guide"

Course: Measurements for Automation and Industrial Production

Prepared by:

ANTONIO PEDICINI (389000080)
MIRIAM PARADISO (389000052)
HAMZA MEKNI (389000063)
OMID MORADIARSHAD (389000067)

Table of contents

Abstract **3**

I Programmable Hardware Instrumentation Used **4**

Power Supply Agilent E36436A . . . pg. 4

NI DAQ USB-6008 data acquisition . . . pg. 5

Arduino UNO R3 . . . pg. 8

II Signal Conditioning Circuit **9**

Circuit Design . . . pg. 9

INA128P . . . pg. 10

Potentiometer . . . pg. 11

Circuit Interconnections . . . pg. 11

III Software **13**

LabVIEW Virtual Instrument . . . pg. 13

Finite State Machine (FSM) LabVIEW Design Pattern . . . pg. 14

Producer-Consumer LabVIEW Design Pattern . . . pg. 15

main.vi . . . pg. 16

Producer Loop . . . pg. 18

Arduino & Power Supply Controls Consumer Loop . . . pg. 20

DAQ Consumer Loop . . . pg. 22

daq_fsm.vi . . . pg. 25

Arduino_FlowPump_Controller.vi . . . pg. 26

Arduino_Serial_FSM.vi . . . pg. 27

Agilent_GPIB_FSM.vi . . . pg. 30

Third Consumer:Current Ramp State Machine . . . pg. 31

External Commands Decoder Loop . . . pg. 34

IV Calibration **35**

Calibration Loop . . . pg. 35

Samples Statistical Distribution . . . pg. 36

Expected Value and Standard Deviation . . . pg. 37

Standard and Expanded Uncertainty . . . pg. 39

Hysteresis Validation . . . pg. 40

Curve Fitting . . . pg. 42

Calibration Curve . . . pg. 43

Absolute Combined Uncertainty . . . pg. 45

Calibration Report . . . pg. 46

V Results **47**

Calibration Dataset Analysis . . . pg. 47

Curves, Expected Values and Standard Deviation . . . pg. 48

Comparison Graph . . . pg. 49

Hysteresis Verification . . . pg. 50

Best-Fit Estimators . . . pg. 51

Calibration Curves . . . pg. 52

Automated Measuring System Uncertainty Budget . . . pg. 53

VI Conclusions **55**

Project Workflow . . . pg. 55

Concluding Remarks . . . pg. 58

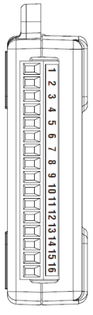
A Arduino Script **59**

Circuit . . . pg. 59

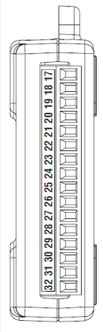
All channels are accessible via removable screw terminals, divided between analog and digital terminals, which allow a quick and flexible connection to signal cables (from 16 to 28 AWG).

The device supports:

- Analog acquisition modes:
  - Single-ended (RSE): all channels measure with respect to GND.
  - Differential (DIFF): measurements on channel pairs, with greater noise immunity.
- External digital trigger (via PFI 0) for measurement synchronization with external events.
- Event counter for counting logic transitions (up to 5 MHz).
- Terminal Assignments

Module	Terminal	Signal, Single-Ended Mode	Signal, Differential Mode
	1	GND	GND
	2	AI 0	AI 0+
	3	AI 4	AI 0–
	4	GND	GND
	5	AI 1	AI 1+
	6	AI 5	AI 1–
	7	GND	GND
	8	AI 2	AI 2+
	9	AI 6	AI 2–
	10	GND	GND
	11	AI 3	AI 3+
	12	AI 7	AI 3–
	13	GND	GND
	14	AO 0	AO 0
	15	AO 1	AO 1
	16	GND	GND

(a) Analog Terminal Assignments

Module	Terminal	Signal
	17	P0.0
	18	P0.1
	19	P0.2
	20	P0.3
	21	P0.4
	22	P0.5
	23	P0 6
	24	P0.7
	25	P1.0
	26	P1.1
	27	P1.2
	28	P1.3
	29	PFI 0
	30	+2.5 V
	31	+5 V
	32	GND

(b) Digital Terminal Assignments

Figure 5

Chapter I

Programmable Hardware Instrumentation Used

Power Supply Agilent E36436A

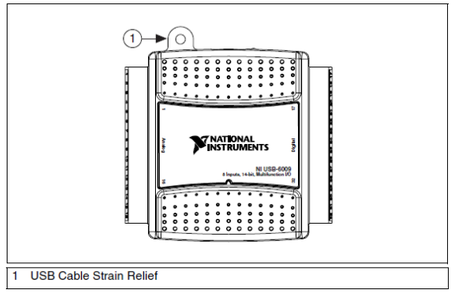
The Agilent E3633A/E3634A DC Power Supply [4] was configured for Constant Voltage (CV) operation, with the output voltage fixed at 12V.

Figure 1: Power Supply E3634A

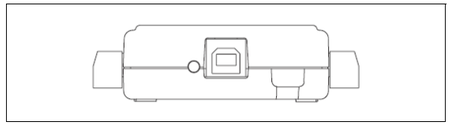
Main characteristics

- **Voltage:** $\pm(\%\text{ of setting} + \text{offset})$
  - **Low Range (0 to 8 V):** $\pm(0.05\% + 5\text{ mV})$
  - **High Range (0 to 20 V):** $\pm(0.05\% + 15\text{ mV})$
- **Current:** $\pm(\%\text{ of setting} + \text{offset})$
  - **Low Range (0 to 20 A):** $\pm(0.2\% + 5\text{ mA})$
  - **High Range (0 to 10 A):** $\pm(0.2\% + 5\text{ mA})$

NI DAQ USB-6008 data acquisition



(a) USB-6008



(b) USB-6008 back view

Figure 2

USB6008 Datasheet[1]

The NI USB-6008 [9] provides connection to eight analog input (AI) channels, two analog output (AO) channels, 12 digital input/output (DIO) channels, and a 32-bit counter with a full-speed USB interface. The following block diagram shows key functional components of the USB-6008:

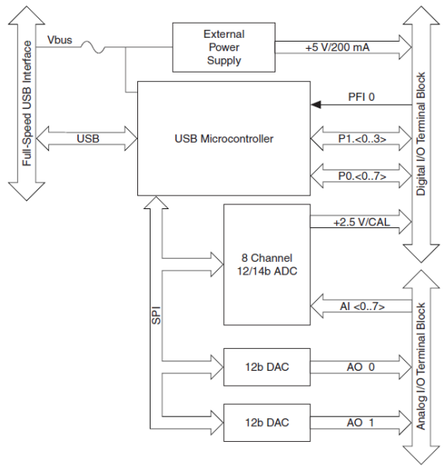


Figure 3

[1] https://www.ni.com/docs/en-US/bundle/usb-6008-specs/page/specs.html?srsltid=AfmBOoomX5uQUUn1iMTOb3gol5ElTAT813--B6ktlUhKk4vixIqbCJly

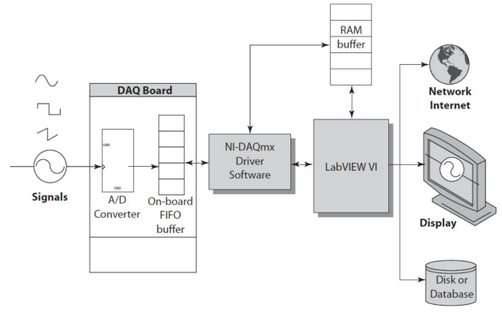


Figure 4

The capabilities of the NI USB-6008 include:

- **Analog Input:** Acquisition of analog signals from external sensors or devices.
- **Analog Output:** Generation of analog signals for driving actuators or simulating processes.
- **Digital I/O:** Reading and writing of TTL digital signals, crucial for control and logic applications.
- **Counter/Timers:** Counting of digital events using a high-resolution 32-bit counter.
- **Power Sourcing:** Provision of regulated +5V and +2.5V outputs to power external low-power devices.

Internal Architecture

The device incorporates:

- A 12-bit ADC that, through a multiplexer (MUX) and a programmable gain amplifier (PGA), allows to selection and acquisition of one of the analog channels in differential or single-ended mode;
- two 12-bit DACs to generate 0–5 V analog signals;
- an integrated USB microcontroller, which manages data transfer, timing and control logic;
- an internal FIFO for data buffering during acquisition.

Arduino UNO R3

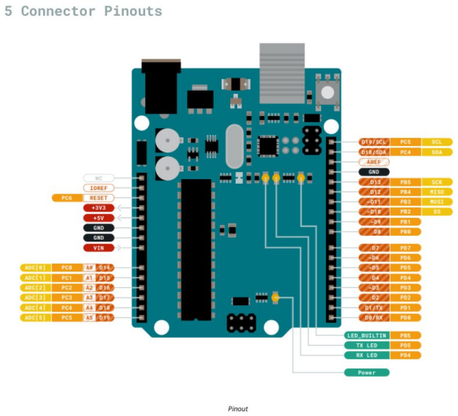


Figure 6: Arduino UNO R3 Pins [5]

Arduino is an open-source platform encompassing a programmable electronic board and an Integrated Development Environment (IDE). The IDE facilitates writing, compiling, and uploading C/C++ based programs (sketches) to the board, leveraging simplified custom libraries and functions like **digitalWrite()** and **analogRead()**, along with a Serial Monitor for real-time communication. A typical Arduino Uno board integrates a central **Microcontroller Unit (MCU)** (e.g., ATmega328P) with built-in memory (Flash, SRAM, EEPROM) to execute code and manage I/O. Its diverse pins include **Digital Input/Output (I/O) Pins** (0–13) for digital control, **PWM Pins** (e.g., 3, 5, 6, 9, 10, 11) for analog simulation, and **Analog Input Pins** (A0–A5) connected to a 10-bit Analog-to-Digital Converter (ADC) for reading variable voltages (0–1023). Essential **Power Pins** (5V, 3.3V, GND, VIN) provide regulated power, while a **USB Port** enables PC connection for code upload, power, and serial communication. A **Reset Button** restarts the MCU, and a **Voltage Regulator** ensures stable internal power. In the context of this Project's work, the Arduino UNO R3 board was used to acquire the digital flow transducer [3] measurements, used as a reference for the calibration procedure. The script that must be installed on the microprocessor is described in the *Appendix A*.

Regarding pin configuration, according to the information reported in the component's datasheet in Figure 8, terminals *1* and *8* are designated for connecting the gain resistor; terminal *7* receives the power supply; the output signal is drawn from terminal *6*; terminals *4* and *5* are connected to a common ground; and terminals *3* and *2* are connected to the signals from which the difference is to be taken.

The gain factor is calculated using the formula provided in the datasheet, where the amplifier's internal feedback resistance is fixed (50 kΩ), while the gain resistance can be varied by connecting an appropriately valued resistor to terminals 1 and 8. During the experimental phases, the gain resistance was set using a potentiometer to the desired value, according to the gain table in the component's datasheet.

Potentiometer

The designed circuit incorporates a Bourns 3296W-1-103LF multi-turn precision potentiometer, employed for fine-tuning the gain of the differential amplifier. The potentiometer is configured as a resistive divider, with its central terminal (also knows as wiper) connected to the amplifier's feedback node. Rotating the trimmer clockwise increases the resistance between the wiper and the CCW terminal, thereby adjusting the gain based on the input signal conditions. Through the potentiometer, the gain resistance was set to 100 Ω, with a tolerance of ±10% as per the device's datasheet. This configuration corresponds to a gain factor of 500. Consequently, if the initial difference between the two signals varies by a few millivolts, the resulting amplified signal will vary between a few hundred millivolts. The potentiometer was connected to the pins *1* and *8* of the used INA128P device.

Devices Interconnections

The conditioning circuit is connected to the measurement system instruments via jumper wires. These interconnections are fully explained in the ***User's Manual***. Hence, please refer to that document for more information. In this section, simple illustrations are provided to the reader, offering a detailed look at an experimental setup. Figure 9a shows the power supplies, stacked one on top of the other. Next, in Figure 9b, the back of the design Conditioning Circuit is shown. This view reveals the complexity of the soldered connections, which are essential for processing and preparing the signals from the sensors before they can be interpreted or used by the system. Then, Figure 10a presents an overview of the

Chapter II

Signal Conditioning Circuit

Circuit Design

The signal conditioning circuit was specifically engineered to amplify the output signals from the analog sensor [2] under calibration. These signals are inherently low-amplitude, rendering them unsuitable for precise detection of airflow variations within the sensor working range. The sensor provides two distinct outputs: a voltage generated by thermopile TP2, exposed to unheated airflow, and a voltage from thermopile TP1, subjected to a temperature variation. The latter exhibits a voltage variation (approximately 15mV at maximum input flow range, as per datasheet specifications) induced by a strategically placed resistor between the two thermopiles. Upon heating, this resistor elevates the temperature of the airflow directed towards the sensor's output, where thermopile TP1 is situated. Consequently, an electrical potential component, attributable to the Seebeck effect, causes a voltage deviation from the no-flow condition.

Nevertheless, experimental measurements using a multimeter corroborate the sensor's datasheet specifications, indicating that the voltages from both thermopiles are in the range of hundreds of millivolts.

Furthermore, to mitigate the risk of overheating the current-controlled air pump, we elected to operate significantly below the sensor's maximum input range, limiting the flow to a maximum of 6 L/min (compared to a specified maximum of 10 L/min). This operational constraint implies that the observed voltage variation will be substantially less than the 15mV indicated in the datasheet.

In essence, the available signals are insufficient to accurately discriminate airflow passage and thus

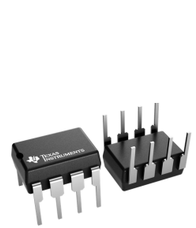
obtain reliable measurements.

Given these limitations, the implementation of a differential amplifier became necessary.

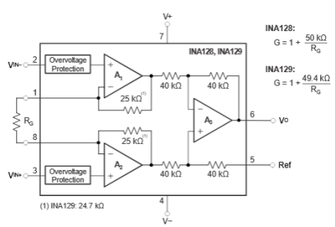
The instrumentation amplifier - INA128P

Fundamentally, the raw signals available are inadequate for reliably discriminating airflow and thus obtaining accurate measurements.

Therefore, the integration of a differential amplifier was deemed essential. We specifically selected the Texas Instruments INA128P differential amplifier.



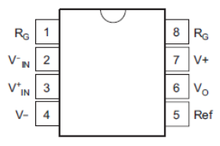
(a)



(b) Simplified Schematic

Figure 7: INA128P package (a) and its corresponding simplified schematic (b).

The INA128 is a low-power, precision instrumentation amplifier, ideally suited for measurement applications demanding high accuracy, minimal drift, and excellent common-mode rejection. Its integrated three-operational amplifier design facilitates a wide range of configurable gains via a single external resistor, thereby ensuring exceptional versatility. Key performance characteristics contributing to the device's efficiency include a low offset voltage (maximum 50 μV), low quiescent current consumption (700 μA), and an outstanding Common Mode Rejection Ratio (CMRR) whose minimum values is 120dB.



(a) Pin Configuration

PIN		TYPE	DESCRIPTION
NAME	NO.		
REF	5	Input	Reference input. This pin must be driven by low impedance or connected to ground.
R_G	1,8	—	Gain setting pin. For gains greater than 1, place a gain resistor between pin 1 and pin 8.
V–	4	Power	Negative supply
V+	7	Power	Positive supply
V_{IN-}	2	Input	Negative input
V_{IN+}	3	Input	Positive input
V_O	6	Output	Output

(b) Pin Functions

Figure 8: Pin INA128P

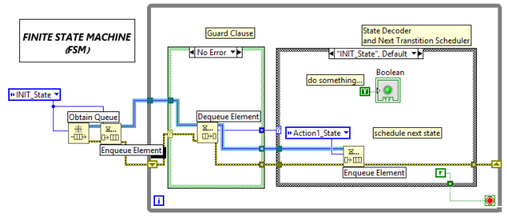


Figure 12: Finite State Machine (FSM) Example Implementation in LabVIEW

Producer-Consumer LabVIEW Design Pattern

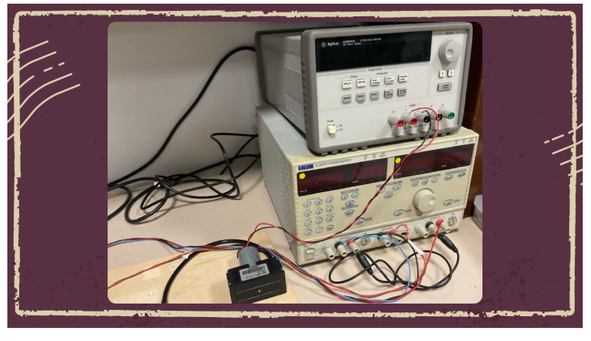
The second paradigm employed, upon which both **main.vi** and the subVI **Arduino_FlowPump_Controller.vi** are based, is the **Producer-Consumer (Event-Driven)** design pattern [7]. This approach is particularly well-suited for real-time applications that must respond promptly to user-generated events through the graphical user interface (GUI), while simultaneously ensuring that the program remains non-blocking—that is, each event is handled asynchronously and independently from others.

This architecture consists of an event-handling structure, referred to as the 'producer', which monitors all relevant user-GUI interactions based on the software's functional requirements. The producer defines the behavioral response for each incoming event, but is unaware of the actual implementation details necessary for event handling. These are delegated to a separate structure known as the 'consumer'. Once the consumer receives a command from the producer, it executes the corresponding operations necessary to complete the event-handling cycle.

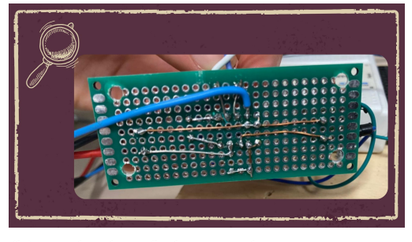
Figure 13 illustrates the block diagram implementing a finite state machine alongside the event-based Producer-Consumer pattern. The events correspond to user-GUI interactions. Each relevant interaction triggers a state transition within the finite state machine, which is implemented in the consumer loop using an enumerated constant to manage state flow.

In this project, a multi-consumer architecture was adopted, where each consumer is designed to respond to events of a different "nature"—that is, events that require access to different software or hardware resources.

automated measuring system's hardware setup. Here we see the conditioning board mounted on a wooden base, connected via cables to the other components, showing how the various elements are physically integrated. Finally, Figure 10b illustrates the electrical schematic of the whole measuring system, where the only element missing is the PC or workstation used and its connections.

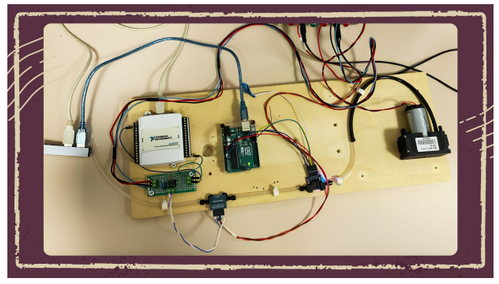


(a) Power Supplies: Agilent E3634A & QL355TP

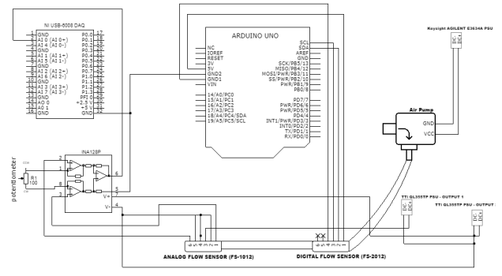


(b) Backside of the Conditioning Circuit Printed Board

Figure 9: Experimental setup components: Power supplies and the backside of the conditioning circuit board.



(a) Physical Measuring System Setup



(b) Schematic of the Whole Measuring System

Figure 10: Overview of the measuring system: Physical setup (left) and detailed schematic (right).

Chapter III

Software

The LabVIEW based Virtual Instrument

The study of the components of the measurement system, along with the delicate task of managing the acquisition of measurement samples simultaneously from multiple devices through different instrument interfaces, led to the choice of the **LabVIEW** development environment. Graphical programming in LabVIEW greatly facilitates the programmer's task, thanks to the wide selection of libraries and software extensions, which offer a high-level abstraction compared to low-level programming of interfaces such as **GPIB** or the **Serial** interface. Furthermore, LabVIEW is a programming environment that was specifically designed to handle the acquisition and generation of input and output signals, respectively, through **DAQ** data acquisition boards.

For these reasons, in order to develop a software system capable of managing the various instruments present in the physical measurement system, we developed several applications (or ***Virtual Instruments (VI)***), each tasked with handling communication with a given device through different programming interfaces.

Figure 11: Icons representing different components (VIs) of the main developed software.

Before any modifications can be made to the program, the reader should be familiar with the concepts of *Enum*, *Queue Operations*, *Event Case Structure*, and *Finite State Machines* as implemented in LabVIEW. Therefore, a brief overview of some of the software paradigms used in this project will be provided, with particular focus on Finite State Machines (FSM) and the Producer-Consumer design pattern.

Finite State Machine (FSM) LabVIEW Design Pattern

A **Finite State Machine** (or FSM) is a computational model used to represent a system that can be in only one state at a time, and that transitions from one state to another in response to specific events or conditions. It is one of the most commonly used paradigms for designing control logic, operational sequences, and reactive systems.

In LabVIEW, a finite state machine can be implemented in various ways [6]. For this project, the FSM has been implemented using a queue-based state scheduling mechanism [7], made possible through the **Synchronization/Queue Operations** *sub-palette*. Figure 12 shows the architecture adopted for all FSMs within this project. Each FSM consists of a **while loop** that iteratively executes the machine, a **Guard Clause**—a case structure that handles potential errors during execution—and a second case structure responsible for decoding and executing the current state's actions, as well as managing the scheduling of the next state.

Outside the while loop of a general Queued-Finite State Machine implementation in LabVIEW, as illustrated in Figure 12, a queue is initialized and used to post state transition commands. The queue's data type is defined as an **Enum** constant, which maps integer values to string labels, thereby abstracting the machine's states into human-readable representations.

The subVIs implementing this FSM paradigm are **Agilent_GPIB_FSM.vi** and **Arduino_Serial_FSM.vi**, which control the logical sequencing and operations required to remotely manage communication with the **Arduino UNO** board and the **Agilent e3634A** power supply, respectively.